



INATEC

Tecnológico Nacional



Taller Luban

INATEC - China

Curso: Fundamentos de Python

Tema:

Generalidades de Python

Contenido:

- Introducción al lenguaje Python
- Comandos básicos
- Condicionales
- Ejercicios prácticos

Docente:

Ing. Fabiola Cerrato

Presentación del lenguaje

Los lenguajes de programación son el puente entre el pensamiento humano y la ejecución de la máquina.

Python es un lenguaje de programación de alto nivel el cual tiene características muy particulares:



¿Qué es tipado?

Es el conjunto de reglas que definen cómo un lenguaje maneja los tipos de datos (números, textos, booleanos) en variables y funciones.

Primeros pasos: Comandos básicos

A continuación, se muestra una guía breve acerca de cómo utilizar el lenguaje:

Python

1. Tipos de datos

En programación, un **tipo de dato** es una regla que clasifica la información. Define el valor que puede tomar una variable, cuánta memoria ocupa y qué operaciones (como sumar o unir) se pueden realizar sobre ella sin causar errores.

Hay tipos de datos que comparte con otros lenguajes; sin embargo, hay tipos únicos como los mostrados en la sección de estructuras.

Nombre	Palabra reservada/Tipo	Sintaxis
Primitivos		
Entero	<code>int</code>	<code>0</code>
Flotante	<code>float</code>	<code>3.14</code>
Cadena de texto	<code>str</code>	<code>"Hola"</code>
Booleano	<code>bool</code>	<code>True/False</code>
Complejo	<code>complex</code>	<code>1 + 2j</code>
Byte	<code>bytes</code>	<code>b"Hola"</code>
Nulo	<code>NoneType</code>	<code>None</code>
Estructuras		
Lista/Pila	<code>list</code>	<code>[1, 2, 3]</code>
Tupla	<code>tuple</code>	<code>(1, 2, 3)</code>
Conjunto	<code>set</code>	<code>{1, 2, 3}</code>
Conj. inmutable	<code>frozenset</code>	<code>frozenset({1, 2, 3})</code>
Diccionario	<code>dict</code>	<code>{"clave": "valor"}</code>
Bytearray	<code>bytearray</code>	<code>bytearray(5)</code>
Rango	<code>range</code>	<code>range(5)</code>
Memory view	<code>memoryview</code>	<code>memoryview(bytes(5))</code>

Ilustración 1: Tipos de datos en Python

Python

2. Declaración de Variables

Una variable en programación es un espacio en la memoria (RAM) del ordenador diseñado para almacenar un valor o dato. Imagínala como una **caja etiquetada** donde guardas información, cuyo contenido (el valor) puede cambiar o ser reutilizado a lo largo de la ejecución de un programa.

En Python las variables deben inicializarse para poder declararse; es decir; no podemos crear una variable sin asignarle un valor; también debemos de tomar en cuenta que no se le asigna el tipo de dato a la variable sino que con la información que guardemos en ella, el compilador se dará cuenta del tipo de dato que será.

Ejemplos:

```
m = 5           <----- variable entera
base = 4.2      <----- variable tipo float
nombre = ' Maria '  <---- variable str
respuesta = True  <----- variable bool
```

Python

3. Imprimir por pantalla

Para imprimir un mensaje en pantalla en Python utilizamos el comando **print**:

Sintaxis:

```
print("mensaje a mostrar")
```

Nota: En *Python* no se utiliza el **punto y coma** para finalizar las líneas de código.

Ejemplo de impresión estándar:

```
print("El robot es amigo del humano")
```

Ejemplo de impresión con formato:

```
nombre_empresa = "YAZAKI"
```

```
print(f"Hola, Bienvenido a la empresa {nombre_empresa}")
```

//Salida en pantalla: Hola, bienvenido a la empresa YAZAKI

Nota: `nombre_empresa` es una variable que ya contiene un valor.

Python

4. Almacenar datos

Para guardar datos ingresados por el usuario en python utilizamos el comando **input**:

Sintaxis:

Variable = **input**("mensaje a mostrar")

Ejemplos:

```
nombre = input('Ingrese su nombre: ')
```

```
edad = int(input('Ingrese su edad: '))
```

```
salario = float(input('Ingrese su salario: '))
```

```
experiencia = input("¿Experiencia? True o False: ")
```

Python

5. Comentarios

Los **Comentarios** se usan para agregar notas explicativas dentro del código. Son ignorados por el compilador y están pensados para la comprensión humana.

En Python tenemos **2** tipos:

a) Comentarios de una línea usando el símbolo **#**:

Ejemplo:

```
# Esto es un comentario
print("¡Hola Mundo!")

print("Imprimiendo en Python")
# Esta línea muestra "Imprimiendo en Python"
```

b) Comentarios de varias líneas usando comillas triples **"""**:

Ejemplo:

```
"""
A continuación, se mostrará
un mensaje en pantalla
"""
print("¡Bienvenido al sistema!")
```

Prácticas comunes de comentarios:

1. Comentarios **TODO**: Para marcar tareas futuras:

Ejemplo:

```
# TODO: hay que optimizar este código
for i in range(1000):
    print(i)
```

2. Desactivar código temporalmente:

Ejemplo:

```
# result = calcular_pago()
result = nueva_calcular_pago()
```

3. Explicar lógica compleja:

Ejemplo:

```
# Calculo del promedio de notas
promedio = (UD1 + UD2 + UD3) / 3
```

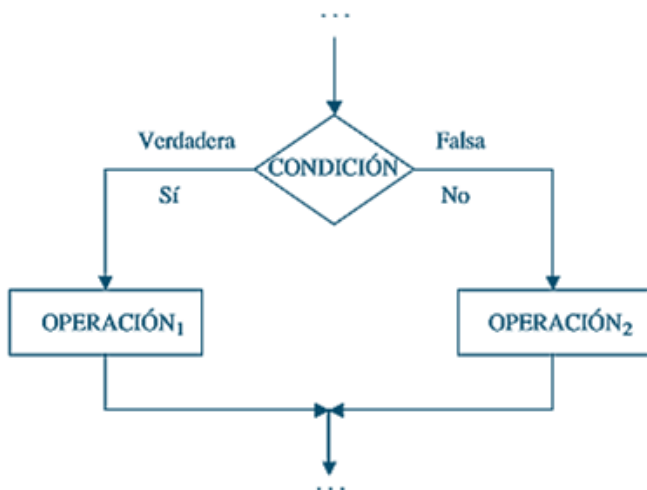
Estructuras Condicionales

Las **estructuras de control** o también llamadas **estructuras de decisión lógica** son utilizadas para seleccionar la ruta más apropiada, la cual se realiza por medio de instrucciones de un algoritmo o también el flujo que debe llevar el control de ejecución cuando se presentan tomas de decisiones.

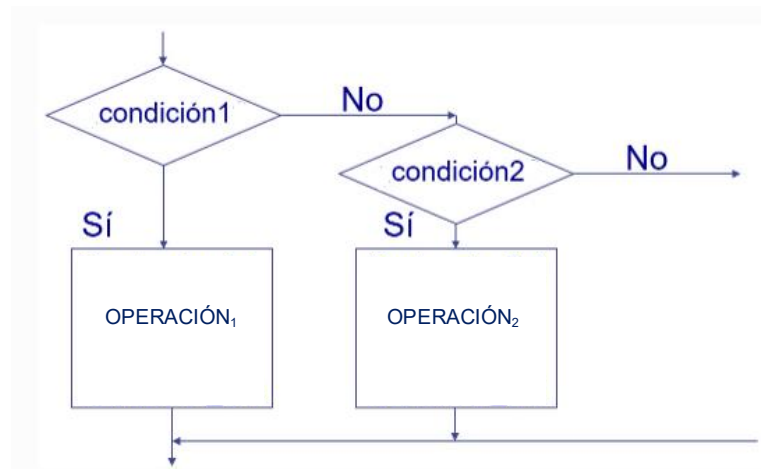
¿Cómo funcionan? - Tipos

Las estructuras de control pueden ser **simple** o **compuesta**, de la cual salen dos posibles caminos:

- ✚ Un conjunto de acciones o secuencias a ejecutar, si el resultado de la condición es **verdadero**.
- ✚ Otro conjunto de acciones o secuencias a realizar, si el resultado de la condición es **falso**.

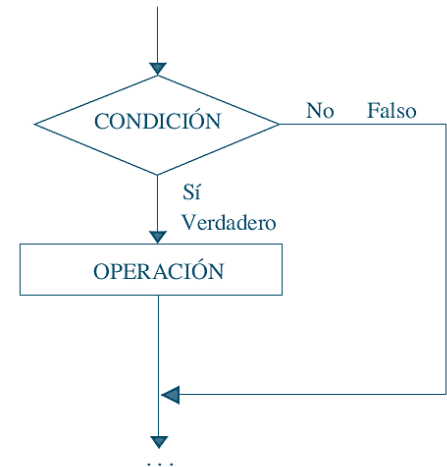


Estructura Simple



Estructura Compuesta

NOTA: Se puede dar el caso de que falte uno de los grupos de instrucciones (operaciones), pero nunca los dos; esto sería un camino nulo que implicaría no tomar ninguna acción.



1. Condicional Si (if):

Se emplea dentro de la programación estructurada para realizar el diseño de un programa a través de una condición que debe o no cumplirse. Se utiliza para representar un algoritmo en programación.

Sintaxis de estructura simple: **if (si)**

```

if <condición>:
    Acción(es)
  
```

Ejemplo:

```

edad = 20
if edad > 18:
    print("Mayor de edad")
  
```

Sintaxis de estructura doble: **if – else (si - sino)****if** <condición>:

Acción(es)

else:

Accion(es)

Ejemplo:

edad = 20

if edad > 18:

print("Mayor de edad")

else:

print("Menor de edad")

Sintaxis de estructura múltiple: **if – elif – else (si – sino si – sino)****if** <condición>:

Acción(es)

elif <condición>:

Accion(es)

....

else:

Accion(es)

Ejemplo #1:

edad = 20

if edad < 13:

print("Niño")

elif edad < 19:

print("Adolescente")

else:

print("Adulto")

Ejemplo #2:

```
edad = 20
if edad < 13:
    print("Niño")
elif edad < 19:
    print("Adolescente")
elif edad < 60:
    print("Adulto")
else:
    print("Anciano")
```

2. Condicional Coincidir (match - case):

La estructura match-case (introducida en Python 3.10) permite evaluar una variable frente a múltiples patrones. Es ideal para simplificar bloques extensos de if-elif.

Sintaxis:

```
match <condición>:
    case <situación 1>: Acción(es)
    case <situación 2>: Acción(es)
    case <situación n>: Acción(es)
    case _: Acción(es)
```

Ejemplo:

hora = 8

match hora:

case 8: print("Es hora del Desayuno")

case 12: print("Es hora del Almuerzo")

case 21: print("Es hora de la Cena")

case _: print("Aún no toca comer")

Nota:

El guion bajo `_` actúa como el caso por defecto (*default*) si no coincide el valor pasado con cualquiera de las condiciones anteriores.

Ejercicios Prácticos

Dado cada uno de los siguientes ejercicios, codifíquelos al lenguaje **Python** utilizando el programa Visual Studio Code.

NIVEL BÁSICO

Ejercicio 1: **Presentación Personal**

Enunciado: Crea un programa que almacene tu nombre, tu edad y tu ciudad en variables. Luego, muestra una presentación como esta: "Hola, me llamo [nombre], tengo [edad] años y vivo en [ciudad]."

Ejercicio 2: **Número positivo, negativo o cero**

Enunciado: Solicita al usuario un número; determina el número ingresado es positivo, negativo o cero

Ejercicio 3: **Calculadora de IMC**

Enunciado: Pida por teclado al usuario su peso en Kg y su altura en mts y calcule y muestre por pantalla su índice de masa corporal (IMC). La fórmula a utilizar es:

$$\text{imc} = \text{peso} / (\text{altura})^2$$

NIVEL INTERMEDIO

Ejercicio 4: Calculadora de operaciones básicas

Enunciado: Crea un programa con menú de opciones donde se le pida al usuario 2 números y elija entre las operaciones de: suma, resta, multiplicación y división. Muestre el resultado de la operación elegida por el usuario por pantalla.

Ejercicio 5: Calculadora de descuentos

Enunciado: Crea un programa que calcule el descuento y total a pagar de un cliente de un supermercado. Si el total a pagar es de menos de C\$1000 no se le aplica descuento, si es entre C\$1000 y C\$5000 tendrá un 15% de descuento y si es más de C\$5000 tendrá un 25% de descuento. Muestre por pantalla el descuento y el total a pagar ya con el descuento incluido.

Ejercicio 6: Clasificador de triángulos

Enunciado: Crea un programa que pida al usuario 3 números los cuales serán 3 lados de un triángulo y muestre por pantalla el tipo de triángulo que es. Por ejemplo: Isósceles, Equilátero, Escaleno o no son lados válidos.

Ejercicio 7: Sistema de recomendación de películas

Enunciado: Crea un programa que pida al usuario un género de película (accion/comedia/ciencia ficción) y la duración (corta/media/larga) y dependiendo de dichas elecciones muestre las siguientes películas:

	Duración		
Género	Corta	Media	Larga
Acción	'John Wick' - 1h 40min	'Mad Max: Fury Road' - 2h	'Avengers: Endgame' - 3h
Comedia	'Superbad' - 1h 50min	'Deadpool' - 2h	'El Gran Lebowski' - 2h 30min
Ciencia Ficción	'Ex Machina' - 1h 50min	'Interestelar' - 2h 50min	'Blade Runner 2049' - 3h

Trabajo Evaluativo (10 pts)

Dado cada uno de los siguientes ejercicios, codifíquelos al lenguaje **Python** utilizando el programa Visual Studio Code.

Ejercicio 1: Calculadora de Edad (3 pts)

Enunciado: Solicita al usuario su año de nacimiento y calcula su edad aproximada (debe pedir también el año actual). Muestra el resultado por pantalla.

Ejercicio 2: Datos de una Mascota (3 pts)

Enunciado: Crea variables para almacenar el nombre de una mascota (String), su edad (int), su peso en kg (double) y si está vacunada (boolean). Luego muestra toda la información por pantalla.

Ejercicio 3: Conversor de divisas (4 pts)

Enunciado: Cree un programa para realizar conversiones de divisas. El programa debe tener un menú de opciones con lo siguiente: Convertir de Córdobas a Dólares, Convertir de Dólares a Córdobas, Convertir de Córdobas a Euros y Convertir de Euros a Córdobas. Pida al usuario la cantidad a convertir y el tipo de cambio. Muestre el resultado de la conversión por pantalla.